B M S COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(Autonomous college under VTU)
P.O. Box No.: 1908, Bull Temple Road, Bangalore-560 019
2026-2027



Alternate Assessment Report on Network Programming

"Performance Analysis of AODV Routing Protocol in a MANET Using NS-3"

with a Proposed Adaptive-Hello Variant (AH-AODV) and a Comparison against OLSR and DSDV

Course Title: Network Programming | Course Code: 23CS7PCNWP

Submitted by

Tanmay Agarwal (1WA23CS010)
Viamrsh Buyanni (1WA23CS039)
Yeruva Taniya Paul (1WA23CS053)
Vijeth MD (1WA23CS041)

Work carried out at

Department of CSE, BMSCE

Faculty in charge

Dr. K. Panimozhi

Associate Professor, Department of Computer Science and Engineering

Date of Submission: 3 November 2026

INDEX

Sl. No.	Chapter Title	Page No.
1	Abstract	1
2	Objectives	2
3	Introduction	3
4	Literature Review	5
5	Research Gap	7
6	Problem Statement	8
7	Methodology and Implementation	9
8	Performance Evaluation	15
9	Output and Results	16
10	Conclusion	21
11	References	22

1. Abstract

This report studies the AODV routing protocol in a mobile ad hoc network (MANET) and proposes AH-AODV (Adaptive-Hello AODV), a variant that reduces AODV's routing overhead. The network was implemented in C++ with the ns-3.48 simulator: nodes move under the Random Waypoint model and use IEEE 802.11b ad-hoc Wi-Fi. A UDP client–server application built on the ns-3 socket API sends constant bit rate traffic, and FlowMonitor and packet traces measure the packet delivery ratio (PDR), throughput, delay, jitter and routing overhead. Compared with the proactive protocols OLSR and DSDV, AODV delivered the most packets, 77.9% on average against 66.2% and 65.0%, and stayed near 75% at 15 m/s while they fell to about 50%, but sent about twice as many control packets, mostly Hello messages broadcast every second. AH-AODV doubles a node's Hello interval, up to 4 s, while its neighbours do not change, and returns to 1 s when they do or when the node carries data. Over 13 settings of density, speed and load (560 runs on shared topologies), AH-AODV sent 31% fewer control packets than AODV, significantly fewer in 10 settings, with the same delivery ratio (78.0% against 77.9%).

Keywords: MANET, AODV, AH-AODV, Hello messages, OLSR, DSDV, ns-3, socket programming, routing overhead.

2. Objectives

2.1 Primary Objective

To implement a mobile ad hoc network in ns-3 in which a UDP client–server application exchanges data over multi-hop routes, to analyse how AODV performs when the number of nodes, the node speed and the traffic load are changed, to compare it with the proactive protocols OLSR and DSDV, and to design and evaluate AH-AODV, an AODV variant that adapts its Hello interval to reduce routing overhead without reducing delivery.

2.2 Specific Objectives

1. To understand socket programming concepts: how a UDP client and server exchange datagrams through calls such as `socket()`, `bind()`, `connect()`, `send()` and `recvfrom()`.
2. To implement UDP client–server communication between two mobile nodes in ns-3, with a constant bit rate (CBR) source on node 0 and a sink on node $N-1$.
3. To study how AODV discovers and repairs multi-hop routes with Route Request, Route Reply and Route Error messages sent over UDP port 654.
4. To design and implement AH-AODV, which adapts each node's Hello interval to the stability of its neighbourhood and to its role on active routes.
5. To configure an IEEE 802.11b ad-hoc network (11 Mbps, 250 m range) with Random Waypoint mobility in a 700 m × 700 m area.
6. To measure the packet delivery ratio, throughput, delay, jitter and packet loss of the data flow, and the routing overhead of each protocol.
7. To analyse network performance while varying the number of nodes, the node speed and the packet rate, and to compare AODV and AH-AODV with each other and with OLSR and DSDV on identical topologies, repeating every setting ten times.
8. To validate the measurements with analytical checks and to report means, standard deviations, 95% confidence intervals and significance tests.

2.3 Expected Outcomes

A reproducible ns-3 program, including the AH-AODV implementation, and scripts that run all 560 simulations, CSV files with one row per run, graphs comparing the four protocols, and an explanation of the observed behaviour in terms of each protocol's mechanisms, together with the limitations of the simulated scenario.

3. Introduction

3.1 Network Programming and the Application

Network programming is the writing of programs that exchange data with programs on other machines. The operating system provides the protocol stack (Wi-Fi or Ethernet, IP, TCP and UDP), and applications reach it through system calls, most commonly the Berkeley (BSD) sockets interface [1]. A socket is a communication endpoint identified by an IP address and a port number: the application writes data into it and the stack delivers the data to the peer socket [2].

The application studied here is multi-hop data delivery in a mobile ad hoc network (MANET). A UDP client on one mobile node sends a stream of packets to a UDP server on another node. There is no access point or wired router: the other mobile nodes forward the packets, and the AODV routing protocol, which itself communicates through UDP sockets, decides which nodes forward them [3, 4].

3.2 Socket Programming, Transport Protocols and System Calls

A UDP application uses a short sequence of system calls. The server calls `socket()` to create a datagram socket, `bind()` to attach it to a known port and `recvfrom()` to receive datagrams. The client calls `socket()`, optionally `connect()` to fix the destination, and `send()` or `sendto()` for every datagram; both call `close()` at the end [1, 5]. TCP adds connection set-up with `listen()`, `accept()` and `connect()`, and provides reliable, ordered delivery with retransmission and congestion control. UDP provides none of these; it only adds an 8-byte header with ports, length and checksum to each datagram [6].

UDP is the right transport for this study for two reasons. First, a constant bit rate stream over UDP is neither slowed down nor repeated by the transport layer, so every lost or delayed packet directly reflects the routing protocol and the wireless channel; TCP would retransmit lost segments and reduce its rate, hiding exactly the effects being measured. Second, AODV itself sends its control messages as UDP datagrams to port 654 [4]. ns-3 provides a socket API modelled on BSD sockets [7], and Section 7.5 shows how the client, the server and AODV use it.

3.3 Reactive Routing and AODV

MANET routing protocols are commonly divided into proactive (table-driven) protocols, such as DSDV [8] and OLSR [9], which keep routes to every destination up to date with periodic control messages, and reactive (on-demand) protocols, such as DSR [10] and AODV [11], which look for a route only when a source has data to send. Reactive proto-

cols avoid maintaining unused routes, at the cost of a discovery delay before the first packets of a flow and after every route break. In ns-3, OLSR sends HELLO messages every 2 s and topology control (TC) messages every 5 s, and DSDV sends full routing updates every 15 s plus triggered updates when routes change.

In AODV, a source without a route broadcasts a Route Request (RREQ). Each node that receives it records a reverse route to the source and rebroadcasts it. The destination, or a node with a fresh enough route, returns a Route Reply (RREP) along the reverse path, which installs the forward route hop by hop. When a link on an active route breaks, a Route Error (RERR) informs the nodes that use it, and the source starts a new discovery. Destination sequence numbers keep routes loop-free and distinguish fresh routes from stale ones [4]. In ns-3, every AODV node also broadcasts a Hello message each second so that its neighbours know the link to it is still up, whether or not anything has changed.

3.4 Tools and Languages

The simulation is written in C++ for the ns-3 discrete-event network simulator, release 3.48 [12, 13], following the ns-3 tutorial [14]. ns-3 supplies the Wi-Fi, mobility, Internet stack, AODV and FlowMonitor models [15], and the program is compiled with ns-3's CMake-based build system and Ninja. Bash scripts run single scenarios and the complete set of experiments; Python 3 with pandas and SciPy processes the CSV results and runs the statistical tests, and matplotlib draws the graphs. The work was done on macOS on Apple Silicon, and the code and results are kept in a Git repository on GitHub [16].

3.5 Motivation

AODV's performance depends on the network around it: a path may not exist in a sparse network, routes break often when nodes move fast, and data and control packets compete for the channel under heavy load. Studying these effects with real devices would need many nodes and repeatable movement. A simulator allows the same scenario to be repeated exactly, one parameter to be changed at a time and every packet to be counted. This project uses that ability to measure AODV under controlled conditions, to compare it fairly with OLSR and DSDV on exactly the same topologies, and to report how much the results vary between random topologies. The comparison showed that AODV's main cost is routing overhead and that most of it comes from its fixed-rate Hello messages; sending fewer of them without losing AODV's reliability is the improvement this project proposes.

4. Literature Review

4.1 MANET Routing Protocols

Abolhasan, Wysocki and Dutkiewicz reviewed MANET routing protocols, grouped them into proactive, reactive and hybrid families, and compared their control overhead and scalability [17]. Perkins and Bhagwat proposed DSDV, which adapts distance-vector routing to mobile nodes by tagging each route with a sequence number issued by the destination, so that nodes prefer fresh routes and avoid loops [8]. Johnson and Maltz proposed DSR, a reactive protocol that discovers routes on demand and carries the complete route in each packet header [10]. Perkins and Royer introduced AODV, which combines DSDV's sequence numbers with on-demand discovery and keeps only hop-by-hop next-hop entries for routes in active use [11]; it was later standardised as RFC 3561 [4].

4.2 Comparative Performance Studies

Broch *et al.* compared DSDV, TORA, DSR and AODV in a packet-level simulation with Random Waypoint mobility and showed that the on-demand protocols kept high delivery ratios at high mobility while DSDV degraded [18]. Johansson *et al.* compared AODV, DSR and DSDV in realistic scenarios such as a conference, event coverage and a disaster area, and found that the relative performance of the protocols depends on the scenario [19]. Perkins, Royer, Das and Marina compared AODV and DSR under varying load, mobility and network size, and found that DSR performed better in less stressful conditions while AODV did better as load and mobility increased [20]. Marina and Das extended AODV to AOMDV, which finds several loop-free, link-disjoint paths in one discovery so that a broken route can be replaced without a new flood [21].

4.3 Mobility and Radio Models

Camp, Boleng and Davies surveyed mobility models for ad hoc network research and showed that the chosen model can change the measured performance of a protocol [22]. Yoon, Liu and Noble showed that the average speed under Random Waypoint decays over time when speeds are drawn from a range that starts at zero [23]; in this project every node moves at one constant speed, so this effect does not arise. Kotz *et al.* tested common simulation assumptions, such as a circular radio range with perfect reception inside it, against measurements from a large outdoor experiment, and showed that they can make simulated results differ substantially from real ones [24].

4.4 Credibility of Simulation Studies and Tools

Kurkowski, Camp and Colagrosso surveyed the simulation studies published at ACM MobiHoc from 2000 to 2005 and found that many did not report enough detail, such as the simulator settings or the number of runs, to be repeated, and they listed common pitfalls [25]. Andel and Yasinsac similarly questioned the credibility of MANET simulations and recommended documenting all parameters and reporting statistically sound results [26]. Riley and Henderson described the design of ns-3, whose C++ models follow real implementations closely and expose a BSD-like socket interface [12], and Carneiro, Fortuna and Ricardo proposed FlowMonitor, which records per-flow packet counts and delays in ns-3 [15].

4.5 Recent Work on AODV

Mammeri classified more than sixty reinforcement-learning-based routing protocols [27], and Quy *et al.* surveyed routing for networks that combine MANETs with the Internet of Things [28]. Alameri *et al.* systematically reviewed modifications to AODV's route discovery, noting that AODV needs few resources because its messages and tables are small [29]. Santana *et al.* used bio-inspired multi-objective algorithms to tune AODV's route recovery and reported lower routing delay, packet loss and route load than standard AODV [30]. Njoroge, Ndia and Mwangi reviewed AODV extensions for performance improvement and found that most modify route discovery, selection or maintenance using quality-of-service parameters, machine learning or signal strength [31].

4.6 Hello Messages and Adaptive Hello Protocols

Chakeres and Belding-Royer showed that whether AODV's Hello messages correctly judge a link to be up depends on the number of Hellos allowed to be lost, the size difference between Hello and data packets and how 802.11b handles broadcasts [32]. Ingelrest, Mitton and Simplot-Ryl proposed a turnover-based adaptive Hello protocol in which each node sets its Hello frequency from the rate at which its neighbourhood changes, keeping neighbour tables accurate with fewer messages than a fixed frequency [33].

4.7 Summary

Routing performance depends strongly on density, mobility, load and the models used, many published simulations are hard to repeat, AODV improvements are judged against standard AODV, and Hellos can be sent less often in a stable neighbourhood. This motivates a repeated-run evaluation of standard AODV and of an adaptive-Hello variant.

5. Research Gap

5.1 Research Gaps Identified

The literature review points to the following gaps:

1. **Scenario dependence.** Protocol performance and even protocol rankings change with mobility, load and network size [18, 19, 20], yet many studies change several factors at once, which hides the cause of a change.
2. **Credibility and repeatability.** Many MANET simulations do not report the settings, runs or confidence needed to repeat or trust them [25, 26].
3. **Single-topology results.** One random topology may connect the source and destination directly and another leave them disconnected, so one run, or a mean without its spread, can mislead.
4. **Idealised models.** A circular radio range and synthetic mobility can make simulated results differ from real deployments [22, 23, 24].
5. **Reliable baselines.** The AODV variants in recent reviews are each compared with standard AODV in their own scenarios [29, 30, 31], so a reproducible standard-AODV baseline is needed to judge them.
6. **Fixed Hello rate.** AODV in ns-3 sends Hellos from every node every 1 s however stable the network is, although adaptive Hello schemes exist [33]; whether an adaptive rule cuts AODV's overhead without losing delivery has not been measured against proactive baselines.

5.2 Selected Research Gap

Gap 4 needs measured traces and is left for future work. This project narrows down to gap 6, using the reproducible baseline of gap 5 and the controls of gaps 1 and 3. The selected research gap is: *AODV's fixed-rate Hellos give it about twice the routing overhead of proactive protocols, and no reproducible evaluation on a current ns-3 release shows whether an adaptive Hello rule removes this overhead without reducing delivery, against AODV, OLSR and DSDV on the same topologies with statistical confidence.*

5.3 Contribution

The project contributes AH-AODV, an adaptive-Hello AODV variant in ns-3.48; a program that runs all four protocols on identical topologies; scripts for 560 reproducible runs; and a paired statistical analysis.

6. Problem Statement

6.1 Problem Statement

Addressing the research gap selected in Section 5.2, the problem is formulated as follows:

> *Design and implement, in ns-3.48, a mobile ad hoc network of N nodes over IEEE 802.11b ad-hoc Wi-Fi, in which a UDP client–server application sends constant bit rate traffic from node 0 to node N−1; propose AH-AODV, an AODV variant that adapts each node's Hello interval to its neighbourhood and its data traffic; and determine, using repeated runs with statistical confidence, whether AH-AODV reduces AODV's routing overhead without reducing its delivery ratio, and how AODV and AH-AODV compare with OLSR and DSDV on the same topologies when the number of nodes, the node speed and the packet rate are varied one at a time.*

6.2 Research Questions

1. How does node density (5 to 20 nodes in 700 m × 700 m) affect delivery and delay?
2. How does node speed (1 to 15 m/s) affect delivery and delay?
3. At what traffic load does the shared channel saturate, and how do throughput, delivery and delay respond?
4. How does AODV compare with OLSR and DSDV in delivery, delay and routing overhead, and which differences are statistically significant?
5. Does AH-AODV send fewer control packets than AODV without lowering the delivery ratio, and in which settings?

6.3 Scope and Assumptions

- All nodes are identical and use the ns-3 default settings of the protocol under test; AH-AODV uses Hello intervals from 1 s (AODV's default) to 4 s and AODV's active route timeout of 3 s.
- The radio range is a hard 250 m cutoff, and there is one data flow, from node 0 to node $N-1$.
- Traffic runs from 2 s to 59 s of a 60 s simulation, so packets in flight at the end are not counted as lost.
- Data metrics cover the data flow only; routing control messages are counted separately as overhead.

7. Methodology and Implementation

7.1 Proposed Architecture

Figure 7.1 shows the architecture of the simulated system. Every node has the same protocol stack: an IEEE 802.11b physical layer and ad-hoc MAC, IPv4 with the routing protocol under test (AODV, AH-AODV, OLSR or DSDV), UDP and the ns-3 socket API. The source runs the CBR client, the destination runs the server, and every other node only routes. A data packet travels down the source's stack, through relays that forward it at the IPv4 layer on routes found by the routing protocol, and up the destination's stack, where FlowMonitor measures it.

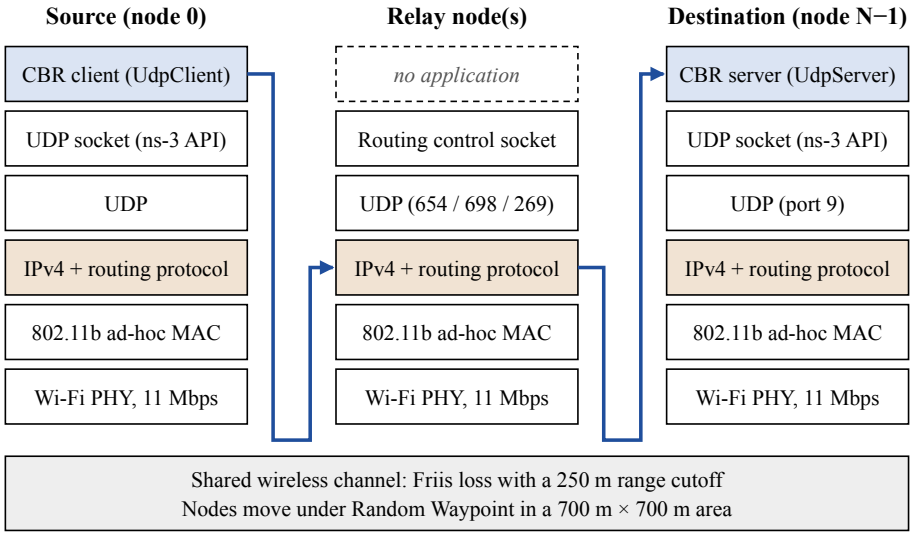


Figure 7.1: Proposed system architecture. Blue arrows show the path of a CBR data packet through one relay.

7.2 Network Communication Model

The network contains N nodes (10 by default) in a 700 m × 700 m square. All nodes use IEEE 802.11b [34] in ad-hoc mode at a fixed 11 Mbps for unicast and broadcast frames. The channel uses a constant-speed propagation delay, the Friis free-space loss model [35] and a 250 m range cutoff. Under the Random Waypoint model each node repeatedly picks a random point in the area, moves to it at the configured speed (5 m/s by default) and pauses for the configured time (0 s by default).

Communication follows a client–server model over UDP. The client on node 0 is connected to the server at 10.1.1.N, port 9, and sends one 512-byte datagram every $1/\lambda$ seconds (4 packets/s by default) from 2 s to 59 s. Data datagrams on port 9 are forwarded by unicast hop by hop, while routing control messages use the protocol's port (AODV and AH-AODV 654, OLSR 698, DSDV 269). Run k uses the same initial positions and move-

ment for all four protocols, so the source and destination may be in direct range, several hops apart or disconnected.

7.3 Mathematical Model

At any time t, the network is represented as a graph

$$G(t) = (V, E(t)), \quad E(t) = \{(u, v) : d_{uv}(t) \le R\},$$

where V is the set of N nodes, $d_{uv}(t)$ is the distance between nodes u and v, and R = 250 m. A packet can be delivered only while a path from the source to the destination exists in $G(t)$ and AODV has discovered it. Let N_{tx} and N_{rx} be the numbers of data packets sent and received, taken from FlowMonitor for the data flow only. The packet delivery ratio, the loss and the throughput over the traffic period T = 57 s, with B_{rx} bytes received, are

$$\text{PDR}(\%) = \frac{N_{\text{rx}}}{N_{\text{tx}}} \times 100,$$

$$N_{\text{loss}} = N_{\text{tx}} - N_{\text{rx}}, \qquad \text{Loss}(\%) = \frac{N_{\text{loss}}}{N_{\text{tx}}} \times 100 = 100 - \text{PDR},$$

$$\text{Throughput(kbps)} = \frac{B_{\text{rx}} \times 8}{T \times 1000}.$$

Throughput is measured at the IP layer, so each 512-byte payload counts as 540 bytes (with 8 bytes of UDP and 20 bytes of IP header); at 100% delivery and 4 packets/s this gives 228 × 540 × 8 / 57 = 17.28 kbps. The average end-to-end delay of the received packets is

$$\overline{D} = \frac{1}{N_{\text{rx}}} \sum_{i=1}^{N_{\text{rx}}} (t_i^{\text{rx}} - t_i^{\text{tx}}),$$

which includes waiting for route discovery, queuing, MAC retransmissions and transmission on every hop. With D_i the delay of the i-th received packet, the average jitter, and the normalised routing load with C routing control packets transmitted by all nodes, are

$$\bar{J} = \frac{1}{N_{\text{rx}} - 1} \sum_{i=2}^{N_{\text{rx}}} |D_i - D_{i-1}|, \qquad \text{NRL} = \frac{C}{N_{\text{rx}}}.$$

In AH-AODV, node u chooses the interval to its next Hello each time its Hello timer expires at time t_k:

$$T_u^{k+1} = \begin{cases} T_0, & \text{if } \Delta_u^k \vee A_u(t_k), \\ \min(2T_u^k, T_{\max}), & \text{otherwise,} \end{cases}$$

where $T_0 = 1$ s is AODV's Hello interval, $T_{\max} = 4$ s, Δ_u^k is true if a neighbour joined or left since the last expiry, and $A_u(t)$ is true if u sent, forwarded or received a unicast packet in the last 3 s. Each Hello advertises a lifetime of $2T$. A node in a stable neighbourhood that carries no data thus sends about 16 Hellos per minute instead of 60.

A scenario is described by $\theta = (N, v, \lambda)$, where v is the node speed and λ the packet rate. For a metric m measured in runs $k = 1, \ldots, K$ ($K = 10$), the mean, the sample standard deviation and the 95% confidence interval are

$$\bar{m}(\theta) = \frac{1}{K} \sum_{k=1}^{K} m(\theta, k),$$

$$s(\theta) = \sqrt{\frac{1}{K-1} \sum_{k=1}^{K} \left(m(\theta, k) - \bar{m}(\theta) \right)^2},$$

$$\mathrm{CI}_{95} = \bar{m}(\theta) \pm t_{0.975,\, K-1} \frac{s(\theta)}{\sqrt{K}},$$

where $t_{0.975,\,9} = 2.262$ is the Student t quantile for ten runs.

7.4 Proposed Algorithm

The proposed protocol, AH-AODV, keeps AODV's route discovery and maintenance and changes only how often each node sends Hello messages. In AODV (Figure 7.2), a source without a route buffers its data and broadcasts a route request (RREQ); every node that forwards it records a reverse route, and the destination, or a node with a fresh route, unicasts a route reply (RREP) back along that path. A node that detects a broken link sends a route error (RERR), and the source discovers the route again [4]. Listing 7.1 gives the AH-AODV Hello rules and Table 7.1 compares the four protocols. Hellos are saved only while they carry no new information, and links on active routes are checked as often as in AODV.

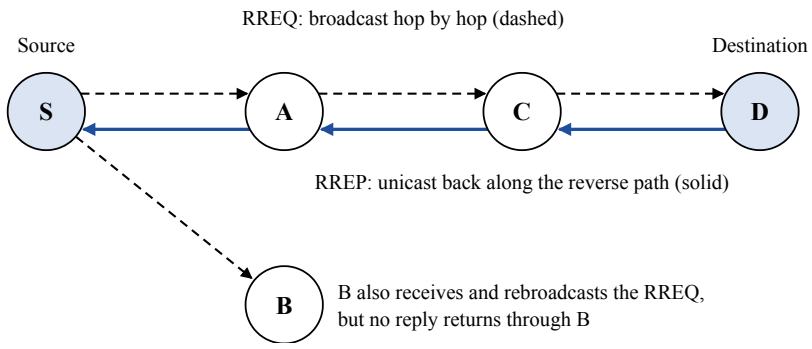


Figure 7.2: AODV route discovery from source S to destination D.

Table 7.1: Routing protocols compared (ns-3 default settings; AH-AODV proposed here) [4, 8, 9]

Protocol	Type	UDP port	How a route is obtained	Control messages
AODV	Reactive, distance vector	654	Discovered on demand; packets wait in a buffer meanwhile	RREQ, RREP, RERR; Hello every 1 s from every node
AH-AODV	Reactive (proposed)	654	As AODV	As AODV, but Hello every 1–4 s, adapted to neighbour changes and data
OLSR	Proactive, link state	698	Always in the table; packets without a route are dropped	HELLO every 2 s; TC every 5 s, flooded by multipoint relays
DSDV	Proactive, distance vector	269	Always in the table; packets wait in a buffer until a route appears	Full update every 15 s; triggered updates on change

Listing 7.1: AH-AODV Hello scheduling at a node (T_0 = 1 s, $T_{\max}$ = 4 s)

```
State: T = T0 (Hello interval in use), changed = false, lastUnicast = −inf
When a neighbour joins or leaves the neighbour table
(first packet heard from it, its Hello lifetime expired, or a MAC failure):
 1. changed = true
 2. If T > T0: set T = T0 and move the next Hello to within T0.
When this node sends, forwards or receives a unicast packet:
 3. lastUnicast = now; if T > T0: set T = T0 and move the next Hello
    to within T0.
When the Hello timer expires:
 4. If changed, or now − lastUnicast < ActiveRouteTimeout (3 s): T = T0
    else T = min(2T, Tmax).
 5. changed = false
 6. Broadcast a Hello (RREP with TTL 1) advertising lifetime 2T, unless an
    AODV broadcast was already sent since the last expiry (as in AODV).
 7. Schedule the next expiry after T.
When a Hello from neighbour n arrives:
 8. Keep n as a neighbour for the lifetime that the Hello advertises.
```

7.5 Socket Programming Implementation

Socket programming appears in two places: the CBR client and server (the ns-3 applications `UdpClient` and `UdpServer`) exchange data through UDP sockets, and the routing agent on every node sends and receives its control messages through its own UDP sockets. ns-3 has no operating-system kernel, so these are C++ calls on simulated sockets rather than system calls, but the API mirrors BSD sockets [7, 12]. Table 7.2 maps each call to its BSD equivalent, and Listing 7.2 shows the calls in the ns-3.48 source of the client, the server and AODV.

The main difference from a BSD program is reception. A BSD server blocks in `recvfrom()`; in a discrete-event simulator nothing may block, so the server registers a callback with `SetRecvCallback()`, which the simulator calls when a packet arrives and which reads the waiting packets with `RecvFrom()`.

Table 7.2: BSD socket calls and their ns-3 equivalents in this project

BSD socket call	ns-3 Socket API	Use in this project
`socket()`	`Socket::CreateSocket(node, UdpSocketFactory)`	Client, server and AODV sockets
`bind()`	`Bind(InetSocketAddress(addr, port))`	Server: port 9; routing: 654 / 698 / 269; client: any free port
`connect()`	`Connect(InetSocketAddress(dst, 9))`	Client fixes the server address 10.1.1.*N*:9
`send()`	`Send(packet)`	Client sends one datagram every 1/λ s
`sendto()`	`SendTo(packet, 0, InetSocketAddress(dst, 654))`	Routing protocols send control messages
`recvfrom()`	`RecvFrom(from)`	Server counts data; routing reads control messages
blocking read / `select()`	`SetRecvCallback(callback)`	Event-driven reception at server and routing
`setsockopt(SO_BROADCAST)`	`SetAllowBroadcast(true)`	RREQ, HELLO and update broadcasts
`setsockopt(SO_BINDTODEVICE)`	`BindToNetDevice(device)`	One routing socket per Wi-Fi interface

Listing 7.2: Socket calls of the client, the server and AODV in ns-3.48 (simplified)

```
// CBR client on node 0 (SourceApplication / UdpClient)
m_socket = Socket::CreateSocket(GetNode(), m_protocolTid); // UDP
m_socket->Bind();                       // any free local port
m_socket->Connect(m_peer);              // server 10.1.1.N, port 9
m_socket->Send(p);                      // one datagram every 1/λ s

// CBR server on node N-1 (SinkApplication / UdpServer)
m_socket->Bind(local);                  // any address, port 9
m_socket->SetRecvCallback(MakeCallback(&UdpServer::HandleRead, this));
while (auto packet = socket->RecvFrom(from)) { ... }  // count packet

// AODV routing agent on every node (aodv-routing-protocol.cc)
socket = Socket::CreateSocket(GetObject<Node>(),
                              UdpSocketFactory::GetTypeId());
socket->SetRecvCallback(MakeCallback(&RoutingProtocol::RecvAodv, this));
socket->BindToNetDevice(l3->GetNetDevice(i));
socket->Bind(InetSocketAddress(iface.GetLocal(), AODV_PORT)); // 654
socket->SetAllowBroadcast(true);
socket->SendTo(packet, 0, InetSocketAddress(destination, AODV_PORT));
```

7.6 Implementation of the Simulation Program

The simulation is a single C++ program, `manet-routing.cc` (about 500 lines), whose parameters, including `--protocol`, are set on the command line; its AODV results are identical run for run to the original `manet-aodv.cc`. Routing overhead is counted with the `Ipv4L3Protocol` "Tx" trace, because FlowMonitor ignores broadcasts: a packet is routing control when its UDP port is the protocol's port. Sent data packets are counted with the client's "Tx" trace, because OLSR drops packets that have no route before FlowMonitor sees them.

AH-AODV is implemented in the `ah-aodv-*.cc` files, a copy of the ns-3.48 AODV model in the namespace `ahaodv`, built next to the unchanged AODV module. Four changes were made (Listing 7.3): the neighbour table reports every neighbour that joins or leaves; the Hello timer applies the rule of Section 7.3; a Hello advertises twice the interval in use; and a receiver keeps the sender as a neighbour for that advertised lifetime. With the attribute `AdaptiveHello` set to false, AH-AODV gives results identical to AODV.

Listing 7.3: AH-AODV Hello timer and neighbour-change handler (simplified)

```
void RoutingProtocol::HelloTimerExpire()
{
    if (m_adaptiveHello) {                        // AH-AODV interval rule
        bool onActiveRoute = m_activeRouteHello &&
            Simulator::Now() - m_lastUnicast < m_activeRouteTimeout;
        m_curHelloInterval = (m_nbChanged || onActiveRoute)
            ? m_helloInterval                     // T0 = 1 s
            : std::min(m_curHelloInterval * 2,    // double, up to
                       m_maxHelloInterval);       // Tmax = 4 s
        m_nbChanged = false;
    }
    SendHello();                          // advertised lifetime = 2 x interval
    m_htimer.Schedule(m_curHelloInterval);
}

void RoutingProtocol::NeighborChanged()  // a neighbour joined or left
{
    m_nbChanged = true;
    ResetHelloInterval();                 // back to 1 s, next Hello within 1 s
}
```

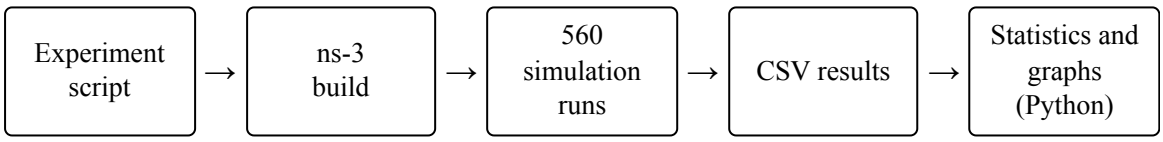


Figure 7.3: Sequence of operations from simulation to graphs (`run_experiments.sh`, `plot_results.py` and `analyse.py`).

8. Performance Evaluation

8.1 Experimental Setup

Simulations ran on macOS 15.6.1 (Apple M2) with ns-3.48 built by Apple clang 16.0.0, and the results were processed with Python 3.13.7, pandas, SciPy and matplotlib. Table 8.1 lists the default parameters and Table 8.2 the three experiments, each of which changes one parameter while the others keep their defaults. Fixed random streams make every run reproducible and give the four protocols identical topologies.

Table 8.1: Simulation parameters (defaults)

Parameter	Value
Number of nodes; area	10; 700 m × 700 m
Wi-Fi standard and mode	IEEE 802.11b, ad-hoc, 11 Mbps (DSSS)
Propagation	Constant-speed delay; Friis loss; 250 m range cutoff
Mobility model	Random Waypoint, 5 m/s, 0 s pause
Network layer and routing	IPv4; AODV, AH-AODV (Hello 1–4 s), OLSR or DSDV
Traffic	UDP client–server, CBR, 512 bytes, 4 packets/s
Simulation time	60 s (traffic from 2 s to 59 s)
Repetitions	Runs 1–10 per setting and protocol: 14 × 4 × 10 = 560 runs

Table 8.2: Experiments and parameter values

Experiment	Parameter varied	Values	Kept fixed
A	Number of nodes	5, 10, 15, 20	5 m/s, 4 packets/s
B	Node speed	1, 5, 10, 15 m/s	10 nodes, 4 packets/s
C	Packet rate	10, 50, 100, 200, 400, 800 pkt/s	10 nodes, 5 m/s

8.2 Performance Metrics

- **Packet delivery ratio (PDR)** and **throughput**: delivered share and rate of the data (higher is better).
- **Delay** and **jitter**: transit time of delivered packets and its variation (lower is better).
- **Packet loss**, including drops at the source for lack of a route (lower is better).
- **Routing overhead**: control packets sent by all nodes, and the NRL (lower is better).

The metrics are defined in Section 7.3. For every setting the mean, sample standard deviation (SD) and 95% confidence interval are reported, and protocols are compared by paired *t*-tests on runs that share a topology, checked with Wilcoxon signed-rank tests ($p < 0.05$ is significant).

9. Output and Results

9.1 Sample Simulation Output

Each run prints its parameters and metrics; Listing 9.1 shows the default scenario with AH-AODV and run 2, whose multi-hop routes break as the nodes move.

Listing 9.1: Output of `./scripts/run_single.sh --protocol=AH-AODV --run=2` (excerpt)

```
AH-AODV MANET Simulation Results (ns-3.48)
  Transmitted packets : 228
  Received packets    : 187
  Dropped at source   : 0 (no route)
  Packet Delivery Ratio : 82.018 %
  Throughput          : 14.173 kbps
  Average E2E delay   : 105.345 ms
  Average jitter      : 81.296 ms
  Control packets     : 505 (24414 bytes)
  Normalised load     : 2.701 control packets per delivered packet
```

On this topology AODV delivered 208 packets (91.2%) with 705 control packets, so AH-AODV saved 200 control packets but delivered 21 fewer. Over the ten default topologies AH-AODV delivered as many packets as AODV in eight, fewer in one and more in one, and sent fewer control packets in all ten.

9.2 Observations

9.2.1 Experiment A: Number of Nodes

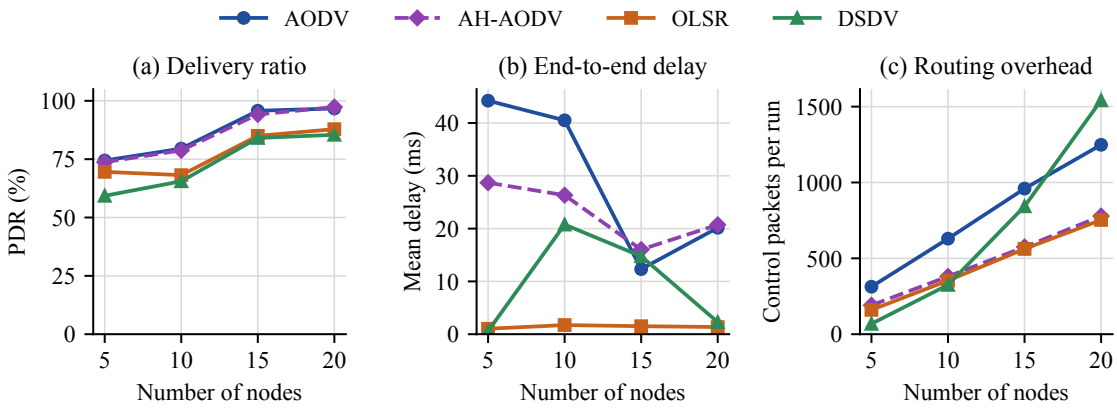


Figure 9.1: Delivery ratio, delay and routing overhead of the four protocols against the number of nodes.

Table 9.1: Effect of the number of nodes on the baseline protocols (mean of 10 runs, PDR as mean ± SD; 5 m/s, 4 packets/s)

Nodes	Protocol	PDR (%)	Throughput (kbps)	Delay (ms)	Jitter (ms)	Control pkts	NRL
5	AODV	74.4 ± 34.9	12.9	44.2	15.5	313	3.79
	OLSR	69.6 ± 32.7	12.0	1.0	0.2	160	1.91
	DSDV	59.3 ± 33.9	10.2	0.6	0.0	68	0.36
10	AODV	79.5 ± 30.2	13.7	40.5	13.8	630	5.53
	OLSR	68.2 ± 31.7	11.8	1.7	0.5	353	3.14
	DSDV	65.5 ± 36.2	11.3	20.8	4.5	325	4.19
15	AODV	95.7 ± 7.7	16.5	12.3	10.2	960	4.42
	OLSR	85.0 ± 8.5	14.7	1.5	0.5	562	2.94
	DSDV	84.1 ± 17.3	14.5	14.8	5.4	843	4.62
20	AODV	96.7 ± 4.2	16.7	20.2	11.1	1,248	5.68
	OLSR	87.9 ± 14.4	15.2	1.4	0.4	753	3.89
	DSDV	85.4 ± 17.4	14.8	2.3	1.2	1,542	8.31

All three protocols deliver more as density rises and partitions become rare: AODV from 74.4% to 96.7%, OLSR from 69.6% to 87.9% and DSDV from 59.3% to 85.4%, with AODV ahead at every density. OLSR's delay stays near 1–2 ms because it drops packets that have no route, whereas AODV (12–44 ms) and DSDV (up to 21 ms) buffer them until a route appears. Overhead grows with the number of nodes, most slowly for OLSR and fastest for DSDV, which exceeds AODV at 20 nodes (1,542 against 1,248 control packets). AH-AODV matches AODV's delivery at every density with 38–40% fewer control packets, about as many as OLSR.

9.2.2 Experiment B: Node Speed

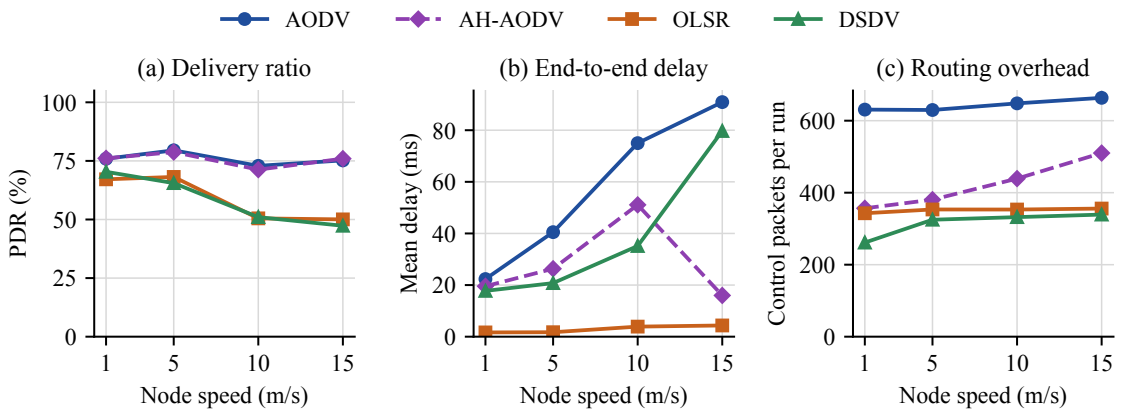


Figure 9.2: Delivery ratio, delay and routing overhead of the four protocols against node speed.

Table 9.2: Effect of node speed on the baseline protocols (mean of 10 runs, PDR as mean ± SD; 10 nodes, 4 packets/s)

Speed (m/s)	Protocol	PDR (%)	Throughput (kbps)	Delay (ms)	Jitter (ms)	Control pkts	NRL
1	AODV	75.9 ± 34.8	13.1	22.3	8.3	631	3.28
	OLSR	67.1 ± 35.4	11.6	1.6	0.1	343	2.19
	DSDV	70.4 ± 33.7	12.2	17.7	10.6	261	1.51
5	AODV	79.5 ± 30.2	13.7	40.5	13.8	630	5.53
	OLSR	68.2 ± 31.7	11.8	1.7	0.5	353	3.14
	DSDV	65.5 ± 36.2	11.3	20.8	4.5	325	4.19
10	AODV	72.9 ± 28.0	12.6	75.0	27.3	648	10.18
	OLSR	50.5 ± 32.9	8.7	3.9	3.0	353	6.78
	DSDV	51.0 ± 32.1	8.8	35.1	8.0	332	6.26
15	AODV	75.2 ± 24.0	13.0	90.9	21.2	664	4.59
	OLSR	50.0 ± 22.4	8.6	4.3	2.9	356	3.75
	DSDV	47.3 ± 20.1	8.2	79.7	17.5	339	3.64

Speed separates the protocols most clearly. AODV keeps its PDR between 72.9% and 79.5% because it repairs broken routes on demand, while OLSR and DSDV fall from 67.1% and 70.4% at 1 m/s to 50.0% and 47.3% at 15 m/s: their tables, refreshed every 2–5 s or 15 s, hold routes that have already broken. AODV pays with delay, which rises from 22.3 ms to 90.9 ms, and with about twice the control packets of the proactive protocols (630–664 against 261–356). AH-AODV keeps AODV's delivery; its saving shrinks from 43% at 1 m/s to 23% at 15 m/s, where neighbours change more often.

9.2.3 Experiment C: Traffic Load

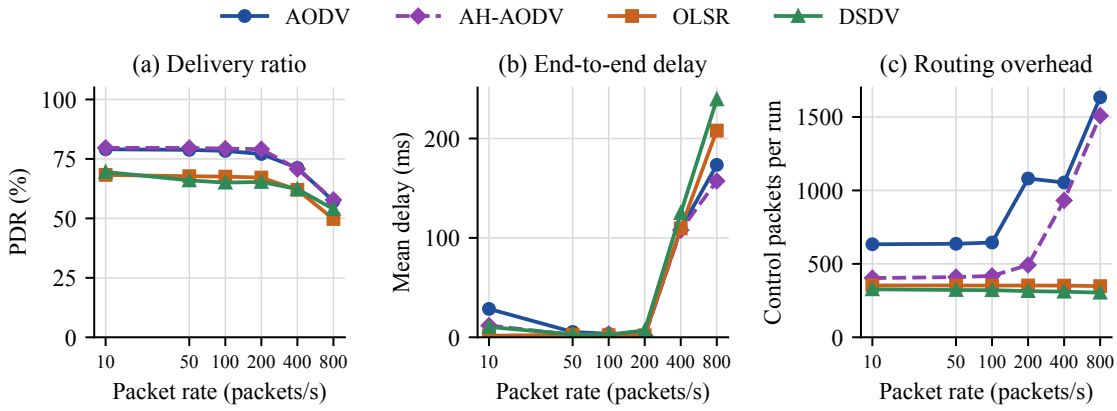


Figure 9.3: Delivery ratio, delay and routing overhead of the four protocols against packet rate (logarithmic axis).

Table 9.3: Effect of traffic load on the baseline protocols (mean of 10 runs, PDR as mean ± SD; 10 nodes, 5 m/s)

Rate (pkt/s)	Protocol	PDR (%)	Throughput (kbps)	Delay (ms)	Jitter (ms)	Control pkts	NRL
10	AODV	79.1 ± 30.3	34.2	28.6	6.8	633	2.26
	OLSR	68.3 ± 31.8	29.5	1.6	0.2	353	1.25
	DSDV	69.5 ± 28.9	30.0	10.4	2.0	327	1.04
50	AODV	78.8 ± 29.9	170.2	5.4	0.6	637	0.45
	OLSR	67.7 ± 32.6	146.3	2.5	0.7	353	0.27
	DSDV	66.0 ± 36.4	142.5	2.9	1.1	322	0.33
100	AODV	78.4 ± 29.8	338.5	3.5	0.2	645	0.23
	OLSR	67.5 ± 32.6	291.8	2.1	0.2	352	0.13
	DSDV	65.0 ± 34.5	281.0	2.8	0.9	321	0.15
200	AODV	77.1 ± 29.3	666.0	2.1	0.1	1,081	0.22
	OLSR	67.2 ± 33.0	580.2	2.0	0.1	353	0.07
	DSDV	65.3 ± 34.9	564.4	6.9	0.4	314	0.07
400	AODV	71.2 ± 31.0	1,230.8	108.9	1.0	1,054	0.09
	OLSR	62.0 ± 35.0	1,071.3	109.6	1.6	352	0.04
	DSDV	62.3 ± 33.4	1,077.1	125.0	0.5	311	0.03
800	AODV	57.2 ± 36.2	1,978.3	173.5	2.6	1,634	0.12
	OLSR	49.7 ± 39.5	1,718.8	208.0	1.9	348	0.04
	DSDV	53.9 ± 36.9	1,863.3	239.4	1.6	304	0.03

Up to 200 packets/s throughput grows in proportion to the load and PDR stays at its low-load level (about 77–79% for AODV and 65–70% for OLSR and DSDV). From 400 packets/s the channel congests; at 800 packets/s PDR falls to 57.2% (AODV), 53.9% (DSDV) and 49.7% (OLSR) and delays rise to 170–240 ms. AODV's control traffic rises with load (633 to 1,634 packets per run) as congestion breaks links and triggers discoveries, while OLSR's and DSDV's stays nearly constant. AH-AODV saves 35–54% up to 200 packets/s but only 8–12% above, where route discoveries dominate.

9.3 Comparative Analysis

Table 9.4 compares AODV and AH-AODV with the proactive baselines over the 13 distinct settings. Single runs ranged from 0% to 100% delivery, so differences are tested by paired *t*-tests on runs that share a topology.

Table 9.4: AODV and AH-AODV compared with OLSR and DSDV over 13 settings (paired *t*-test, $p < 0.05$)

Measure	AODV vs OLSR	AODV vs DSDV	AH-AODV vs OLSR	AH-AODV vs DSDV
Mean PDR	77.9% vs 66.2%	77.9% vs 65.0%	78.0% vs 66.2%	78.0% vs 65.0%
PDR significantly higher	12 of 13	9 of 13	11 of 13	9 of 13
Mean control packets	829 vs 384	829 vs 431	569 vs 384	569 vs 431
Control packets significantly more	12 of 13	11 of 13 (fewer at 20 nodes)	6 of 13	11 of 13 (fewer at 15 and 20 nodes)

AODV delivered the most data, 3 to 28 points more per setting and never significantly less: it repairs routes when they are needed, while OLSR and DSDV keep stale routes between updates, and it holds packets during discovery, whereas OLSR drops them (72–75% of OLSR's losses). The price is about twice the control packets and a higher delay and jitter. AH-AODV keeps the delivery advantage with 569 control packets per run, close to OLSR's and DSDV's, and below DSDV's at 4 packets/s (462 against 530). This agrees with earlier studies in which on-demand protocols delivered more under mobility and load [18, 20].

9.4 Effect of the Proposed AH-AODV

Table 9.5: AH-AODV compared with AODV (10 paired runs per setting; paired *t*-tests; * default)

Setting	PDR (%) AODV	PDR (%) AH-AODV	PDR *p*	Control AODV	Control AH-AODV	Control change	Control *p*
5 nodes	74.4	73.7	0.343	313	190	−39%	< 0.001
10 nodes*	79.5	78.8	0.456	630	381	−40%	< 0.001
15 nodes	95.7	94.2	0.368	960	575	−40%	< 0.001
20 nodes	96.7	97.3	0.531	1,248	779	−38%	< 0.001
1 m/s	75.9	76.1	0.343	631	357	−43%	< 0.001
10 m/s	72.9	71.3	0.257	648	439	−32%	< 0.001
15 m/s	75.2	76.0	0.803	664	510	−23%	< 0.001
10 pkt/s	79.1	79.7	0.633	633	403	−36%	< 0.001
50 pkt/s	78.8	79.6	0.435	637	410	−36%	< 0.001
100 pkt/s	78.4	79.3	0.327	645	418	−35%	< 0.001
200 pkt/s	77.1	79.1	0.606	1,081	492	−54%	0.158
400 pkt/s	71.2	70.8	0.667	1,054	932	−12%	0.345
800 pkt/s	57.2	57.8	0.745	1,634	1,509	−8%	0.625

AH-AODV sent fewer control packets than AODV in every setting, 31% fewer on average (829 to 569 per run) and significantly fewer in 10 of 13 settings ($p < 0.001$; at 200 packets/s only by the Wilcoxon test, $p = 0.004$). PDR and throughput differed significantly in no setting, and the mean delay fell from 48.3 ms to 35.6 ms (significant only at 200 packets/s). Without the active-route rule, a pilot at 15 m/s gave a PDR of 71.1% instead of 76.0%.

9.5 Validation of the Proposed Approach

All 560 runs passed the measurement checks: packets sent always equalled 57 s × λ, loss equalled 100 − PDR, throughput matched offered load × PDR, and OLSR's overhead matched its timers. The program reproduces all 140 runs of the original AODV study, and with `--adaptiveHello=false` AH-AODV matched AODV in all 15 test runs, so the differences in Table 9.5 come from the Hello rule alone.

10. Conclusion

Findings. A MANET was implemented in ns-3.48 in which a UDP client–server application sends CBR traffic, and AODV was compared with OLSR and DSDV on shared topologies. AODV delivered 77.9% of packets on average (a packet loss of 22.1%) against 66.2% and 65.0%, significantly more in 12 and 9 of 13 settings and never significantly less, with its largest advantage under mobility (75.2% against 50.0% and 47.3% at 15 m/s). Its cost was about twice the routing overhead, most of it Hello messages, and a higher delay and jitter (40.5 ms and 13.8 ms against 1.7 ms and 0.5 ms for OLSR in the default scenario).

Proposed AH-AODV. AH-AODV adapts each node's Hello interval between 1 s and 4 s to changes in its neighbourhood and keeps 1 s on nodes that carry data. On the same topologies it sent 31% fewer control packets than AODV, significantly fewer in 10 of 13 settings, with no significant change in delivery ratio or throughput in any setting (78.0% against 77.9%), a lower mean delay (35.6 ms against 48.3 ms) and the same significant delivery advantage over OLSR and DSDV. Its saving is largest in slow or lightly loaded networks and small under heavy load, where route discoveries dominate the overhead.

Contribution to the research gap. The project shows reproducibly, with paired statistics on a current ns-3 release, that an adaptive Hello rule removes most of AODV's overhead penalty against proactive protocols without reducing delivery, which is the gap selected in Chapter 5.

Limitations. The radio model uses a hard 250 m cutoff, only one CBR flow was simulated, Random Waypoint is synthetic, all protocols use default settings with traffic starting at 2 s, ten runs are too few for the density and speed effects within AODV to be significant, and AH-AODV's 4 s maximum was chosen from a pilot of 2, 3, 4 and 8 s rather than tuned for each scenario.

Future work. Test AH-AODV with many flows and a fading radio model, where Hellos are lost more often; combine it with congestion-aware route discovery to cut the overhead under heavy load; tune the protocol timers and add a warm-up period; add TCP traffic and more runs per setting; and measure energy use and resilience to attacks such as the black hole attack.

11. References

[1] W. R. Stevens, B. Fenner, and A. M. Rudoff, *UNIX Network Programming, Volume 1: The Sockets Networking API*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2003.

[2] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Hoboken, NJ, USA: Pearson, 2021.

[3] S. Corson and J. Macker, "Mobile Ad hoc Networking (MANET): Routing Protocol Performance Issues and Evaluation Considerations," RFC 2501, IETF, Jan. 1999. Available: https://www.rfc-editor.org/rfc/rfc2501.

[4] C. E. Perkins, E. M. Belding-Royer, and S. R. Das, "Ad hoc On-Demand Distance Vector (AODV) Routing," RFC 3561, IETF, Jul. 2003. Available: https://www.rfc-editor.org/rfc/rfc3561.

[5] The Linux man-pages project, "socket(2), bind(2), connect(2), send(2), recv(2), close(2) and udp(7)," *Linux Programmer's Manual*. Available: https://man7.org/linux/man-pages/. Accessed Oct. 2026.

[6] J. Postel, "User Datagram Protocol," RFC 768, IETF, Aug. 1980. Available: https://www.rfc-editor.org/rfc/rfc768.

[7] ns-3 Project, "Sockets APIs," *ns-3 Model Library*, online documentation. Available: https://www.nsnam.org/docs/models/html/sockets-api.html. Accessed Oct. 2026.

[8] C. E. Perkins and P. Bhagwat, "Highly Dynamic Destination-Sequenced Distance-Vector Routing (DSDV) for Mobile Computers," in *Proc. ACM SIGCOMM*, 1994, pp. 234–244.

[9] T. Clausen and P. Jacquet, "Optimized Link State Routing Protocol (OLSR)," RFC 3626, IETF, Oct. 2003. Available: https://www.rfc-editor.org/rfc/rfc3626.

[10] D. B. Johnson and D. A. Maltz, "Dynamic Source Routing in Ad Hoc Wireless Networks," in *Mobile Computing*, T. Imielinski and H. F. Korth, Eds. Boston, MA, USA: Kluwer Academic, 1996, pp. 153–181.

[11] C. E. Perkins and E. M. Royer, "Ad-hoc On-Demand Distance Vector Routing," in *Proc. 2nd IEEE Workshop on Mobile Computing Systems and Applications (WMCSA)*, 1999, pp. 90–100.

[12] G. F. Riley and T. R. Henderson, "The ns-3 Network Simulator," in *Modeling and Tools for Network Simulation*, K. Wehrle, M. Güneş, and J. Gross, Eds. Berlin, Germany: Springer, 2010, pp. 15–34.

[13] ns-3 Project, "ns-3 Manual and Model Library (AODV, Wi-Fi, Mobility and FlowMonitor)," online documentation. Available: https://www.nsnam.org/documentation/. Accessed Oct. 2026.

[14] ns-3 Project, "ns-3 Tutorial," online tutorial. Available: https://www.nsnam.org/docs/tutorial/html/. Accessed Oct. 2026.

[15] G. Carneiro, P. Fortuna, and M. Ricardo, "FlowMonitor: A Network Monitoring Framework for the Network Simulator 3 (NS-3)," in *Proc. 4th Int. ICST Conf. Performance Evaluation Methodologies and Tools (VALUETOOLS)*, 2009.

[16] T. Agarwal, V. Buyanni, Y. T. Paul, and Vijeth MD, "manet-routing-comparison: AODV, AH-AODV, OLSR and DSDV in a MANET Using ns-3," source code and results. Available: https://github.com/vijethmd/manet-routing-comparison.

[17] M. Abolhasan, T. Wysocki, and E. Dutkiewicz, "A Review of Routing Protocols for Mobile Ad Hoc Networks," *Ad Hoc Networks*, vol. 2, no. 1, pp. 1–22, 2004.

[18] J. Broch, D. A. Maltz, D. B. Johnson, Y.-C. Hu, and J. Jetcheva, "A Performance Comparison of Multi-Hop Wireless Ad Hoc Network Routing Protocols," in *Proc. ACM MobiCom*, 1998, pp. 85–97.

[19] P. Johansson, T. Larsson, N. Hedman, B. Mielczarek, and M. Degermark, "Scenario-based Performance Analysis of Routing Protocols for Mobile Ad-hoc Networks," in *Proc. ACM*

MobiCom, 1999, pp. 195–206.

[20] C. E. Perkins, E. M. Royer, S. R. Das, and M. K. Marina, “Performance Comparison of Two On-Demand Routing Protocols for Ad Hoc Networks,” *IEEE Personal Communications*, vol. 8, no. 1, pp. 16–28, Feb. 2001, doi: 10.1109/98.904895.

[21] M. K. Marina and S. R. Das, “On-demand Multipath Distance Vector Routing in Ad Hoc Networks,” in *Proc. IEEE Int. Conf. Network Protocols (ICNP)*, 2001, pp. 14–23.

[22] T. Camp, J. Boleng, and V. Davies, “A Survey of Mobility Models for Ad Hoc Network Research,” *Wireless Communications and Mobile Computing*, vol. 2, no. 5, pp. 483–502, 2002.

[23] J. Yoon, M. Liu, and B. Noble, “Random Waypoint Considered Harmful,” in *Proc. IEEE INFOCOM*, 2003.

[24] D. Kotz, C. Newport, R. S. Gray, J. Liu, Y. Yuan, and C. Elliott, “Experimental Evaluation of Wireless Simulation Assumptions,” in *Proc. ACM/IEEE Int. Symp. Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM)*, 2004, pp. 78–82, doi: 10.1145/1023663.1023679.

[25] S. Kurkowski, T. Camp, and M. Colagrosso, “MANET Simulation Studies: The Incredibles,” *ACM SIGMOBILE Mobile Computing and Communications Review*, vol. 9, no. 4, pp. 50–61, 2005.

[26] T. R. Andel and A. Yasinsac, “On the Credibility of MANET Simulations,” *IEEE Computer*, vol. 39, no. 7, pp. 48–54, Jul. 2006.

[27] Z. Mammeri, “Reinforcement Learning Based Routing in Networks: Review and Classification of Approaches,” *IEEE Access*, vol. 7, pp. 55916–55950, 2019, doi: 10.1109/ACCESS.2019.2913776.

[28] V. K. Quy, V. H. Nam, D. M. Linh, and L. A. Ngoc, “Routing Algorithms for MANET-IoT Networks: A Comprehensive Survey,” *Wireless Personal Communications*, vol. 125, no. 4, 2022, doi: 10.1007/s11277-022-09722-x.

[29] I. Alameri, J. Komarkova, T. Al-Hadhrami, and A. Lotfi, “Systematic Review on Modification to the Ad-hoc On-Demand Distance Vector Routing Discovery Mechanics,” *PeerJ Computer Science*, vol. 8, art. e1079, 2022, doi: 10.7717/peerj-cs.1079.

[30] C. Santana, M. Macedo, E. Alves, M. T. Guerreiro, H. V. Siqueira, A. Gokhale, and C. Bastos-Filho, “Bio-Inspired Multi-Objective Algorithms Applied on the Optimization of the AODV’s Routing Recovery Mechanism,” *IEEE Access*, vol. 11, pp. 116480–116496, 2023, doi: 10.1109/ACCESS.2023.3322691.

[31] P. M. Njoroge, J. G. Ndia, and P. M. Mwangi, “A Systematic Review of Ad Hoc On-Demand Distance Vector Routing Protocol for Performance Improvement in Mobile Ad Hoc Networks,” *International Journal of Wireless & Mobile Networks*, vol. 17, no. 2, Apr. 2025, doi: 10.5121/ijwmn.2025.17201.

[32] I. D. Chakeres and E. M. Belding-Royer, “The Utility of Hello Messages for Determining Link Connectivity,” in *Proc. 5th Int. Symp. Wireless Personal Multimedia Communications (WPMC)*, Honolulu, HI, USA, Oct. 2002, vol. 2, pp. 504–508.

[33] F. Ingelrest, N. Mitton, and D. Simplot-Ryl, “A Turnover based Adaptive HELLO Protocol for Mobile Ad Hoc and Sensor Networks,” in *Proc. 15th IEEE Int. Symp. Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, Istanbul, Turkey, Oct. 2007, pp. 9–14.

[34] IEEE Std 802.11b-1999, “Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications: Higher-Speed Physical Layer Extension in the 2.4 GHz Band,” IEEE, 1999.

[35] H. T. Friis, “A Note on a Simple Transmission Formula,” *Proceedings of the IRE*, vol. 34, no. 5, pp. 254–256, 1946.